

variable
scope

CSCI
134

what does this code display on the screen?

script.py

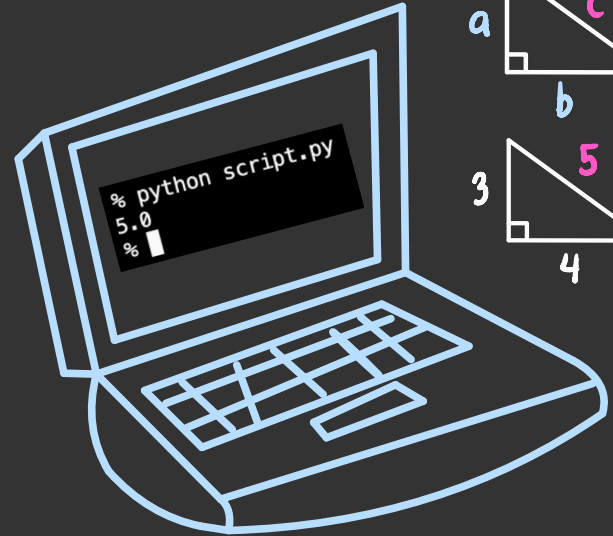
```
a = 3
b = 4
square = lambda x: x*x
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



what does this code display on the screen?

script.py

```
a = 3
b = 4
square = lambda x: x*x
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



what if we make this change?

script.py

```
a = 3
b = 4
square = lambda a a a
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



nothing changes

script.py

```
a = 3
b = 4
square = lambda a a a
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



what if we make **this** change?

script.py

```
a = 3
b = 4
square = lambda a, b: a * b
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



not the same output

script.py

```
a = 3
b = 4
square = lambda a, b: a * b
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



but also not an error

script.py

```
a = 3
b = 4
square = lambda a, a, b: a * b
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



when we reuse variable names, how does python know which value we want?

script.py

```
a = 3
b = 4
square = lambda a: a*b
c = square(a) + square(b)
c = pow(c, 0.5)
print(c)
```



how do

you

do it?



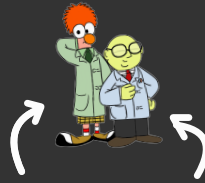
moving from a big city to williamstown
requires a social adjustment



elmo



zoe



beaker bunsen



oscar



bert



ernie



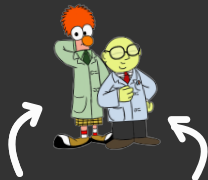
but python is more like a modern american city where nobody knows their neighbors



elmo



zoe



beaker bunsen



oscar



bert



ernie



oscar



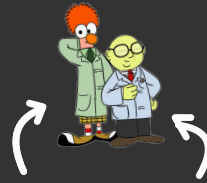
though everyone knows oscar the grouch
because they pass him every day



elmo



zoe



beaker bunsen



oscar



when the name "oscar" is mentioned in these houses,
the residents assume it refers to oscar the grouch



oscar



bert

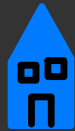


ernie

oscar



but when the name "oscar" is mentioned in this house,
the residents assume it refers to oscar wilde



```
oscar = 333444333 # oscar the grouch's social security number
```

```
def house1():
```

```
|   elmo = 123456789 # elmo's social security number
```

```
|   zoe = 421231523  # zoe's social security number
```

```
def house2():
```

```
|   beaker = 77777777 # beaker's social security number
```

```
|   bunsen = 392781243 # bunsen's social security number
```

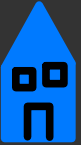
```
def house3():
```

```
|   oscar = 567891234 # oscar wilde's social security number
```

```
|   bert = 998877665  # bert's social security number
```

```
|   ernie = 112233445 # ernie's social security number
```

in python, the "street" is the left-side of the screen



sesame.py

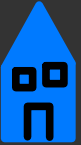
```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)  
    print(zoe)  
    print(oscar)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(beaker)  
    print(bunsen)  
    print(oscar)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(bert)  
    print(ernie)  
    print(oscar)
```

these functions have
no local variable named
oscar



sesame.py

```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)  
    print(zoe)  
    print(oscar)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(beaker)  
    print(bunsen)  
    print(oscar)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(bert)  
    print(ernie)  
    print(oscar)
```

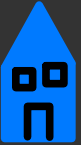
so they assume that
oscar refers to the
variable at street level
(the "global scope")

```
In [2]: house1()  
123456789  
421231523  
333444333
```



```
In [3]: house2()  
777777777  
392781243  
333444333
```





sesame.py

```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)  
    print(zoe)  
    print(oscar)
```

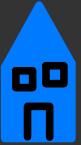
```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(beaker)  
    print(bunsen)  
    print(oscar)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(bert)  
    print(ernie)  
    print(oscar)
```

this function does have
a local variable named

oscar





sesame.py

```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)  
    print(zoe)  
    print(oscar)
```

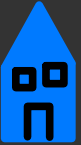
```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(beaker)  
    print(bunsen)  
    print(oscar)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(bert)  
    print(ernie)  
    print(oscar)
```

so it assumes that
oscar refers to
the local variable
(the "**local scope**")

```
In [4]: house3()  
998877665  
112233445  
567891234
```





sesame.py

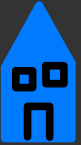
```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(elmo)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(elmo)
```

let's alter
the code
a bit



sesame.py

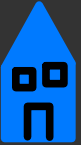
```
oscar = 333444333
```

```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(elmo)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(elmo)
```

← this function has a
local variable named
elmo



```
sesame.py
```

```
oscar = 333444333
```

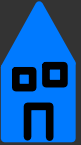
```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(elmo)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(elmo)
```

so it assumes that
elmo refers to
the local variable
(the "**local scope**")

```
In [2]: house1()  
123456789
```



sesame.py

```
oscar = 333444333
```

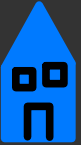
```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(elmo)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(elmo)
```

these functions have
no local variable named
elmo





sesame.py

```
oscar = 333444333
```



```
def house1():
```

```
    elmo = 123456789
```

```
    zoe = 421231523
```

```
    print(elmo)
```

```
def house2():
```

```
    beaker = 777777777
```

```
    bunsen = 392781243
```

```
    print(elmo)
```

```
def house3():
```

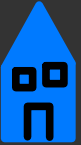
```
    oscar = 567891234
```

```
    bert = 998877665
```

```
    ernie = 112233445
```

```
    print(elmo)
```

but there is also
no **elmo** in the
global scope



sesame.py

```
oscar = 333444333
```



```
def house1():  
    elmo = 123456789  
    zoe = 421231523  
    print(elmo)
```

```
def house2():  
    beaker = 777777777  
    bunsen = 392781243  
    print(elmo)
```

```
def house3():  
    oscar = 567891234  
    bert = 998877665  
    ernie = 112233445  
    print(elmo)
```

this causes an
error

```
In [3]: house2()
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 house2()
```

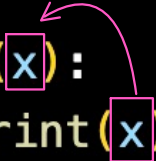
```
File ~/Documents/projects/teaching/csc101/04/04_sesame.py:11  
11 beaker = 777777777  
12 bunsen = 392781243  
----> 13 print(elmo)
```

```
NameError: name 'elmo' is not defined
```

scope.py

```
x = 4
```

```
def f(x):  
    print(x)
```

A diagram consisting of two small red squares, one around the 'x' in the function definition 'def f(x):' and another around the 'x' in the function call 'print(x)'. A red curved arrow points from the 'x' in 'print(x)' to the 'x' in 'def f(x):', indicating that the argument 'x' is resolved to the local variable 'x' within the function's scope.

```
In [1]: from scope import f
```

```
In [2]: f(6)  
6
```

iPython

function arguments are treated like
local variables

scope.py

w = 1

x = 2

y = 3

def f(x):

w = 5

print(w)

print(x)

print(y)

print(z)

?

w

x

y

z

```
In [1]: from scope import f
```

```
In [2]: f(4)
```

5

4

3

NameError

Traceback (most recent call last)

Cell In[2], line 1

----> 1 f(4)

File ~/Documents/projects/teaching/
csci134/spring2025/code/scope.py:10

, in f(x)

8 print(x)

9 print(y)

----> 10 print(z)

NameError: name 'z' is not defined

iPython

a visual summary